

SOFTWARE DESIGN DOCUMENT

Hotel Management System

WinForms Desktop Edition

Document Version	1.0
Date	2026-05-21
Prepared by	Abdurahman Ibrahim
Reference SRS	Software Requirements Specification, Hotel Management System, v1.2 (2026-05-11)
Standard Reference	IEEE Std 1016-2009 — Standard for Information Technology — Systems Design — Software Design Descriptions

Table of Contents

If this page appears blank, right-click below and select "Update Field" (or press F9) to populate the table of contents.

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the internal design of the **Hotel Management System (HMS)** — a single-user Windows desktop application that implements the requirements stated in the SRS v1.2. It is intended for:

- The **development team** maintaining or extending the code.
- The **course instructor** evaluating compliance with IEEE 1016-2009.
- **QA / Test Engineers** mapping test cases to design elements.
- **Future contributors** reading the source after the original author has moved on.

The SDD answers "how the system is built"; the SRS answers "what the system shall do." The two are mutually exclusive: nothing prescriptive about user behaviour appears here, and nothing prescriptive about source files appears in the SRS.

1.2 Scope

The SDD covers the entire v1.2 release: every namespace, every service, every model, and every form folder shipped with commit b11b95b on branch main. Out of scope: any design slated for v1.3 or later (persistence layer, payment gateway, multi-user mode).

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
HMS	Hotel Management System — the product.
SRS	Software Requirements Specification (companion document).
SDD	Software Design Document (this document).
DI	Dependency Injection.
DTO	Data Transfer Object.
POCO	Plain Old CLR Object.
Onion architecture	A layered architecture where outer rings depend on inner rings but never the reverse.
WinForms	Microsoft Windows Forms — the UI toolkit.
<code>BindingList<T></code>	A System.ComponentModel list type whose mutations raise ListChanged events; bound WinForms controls re-render automatically.

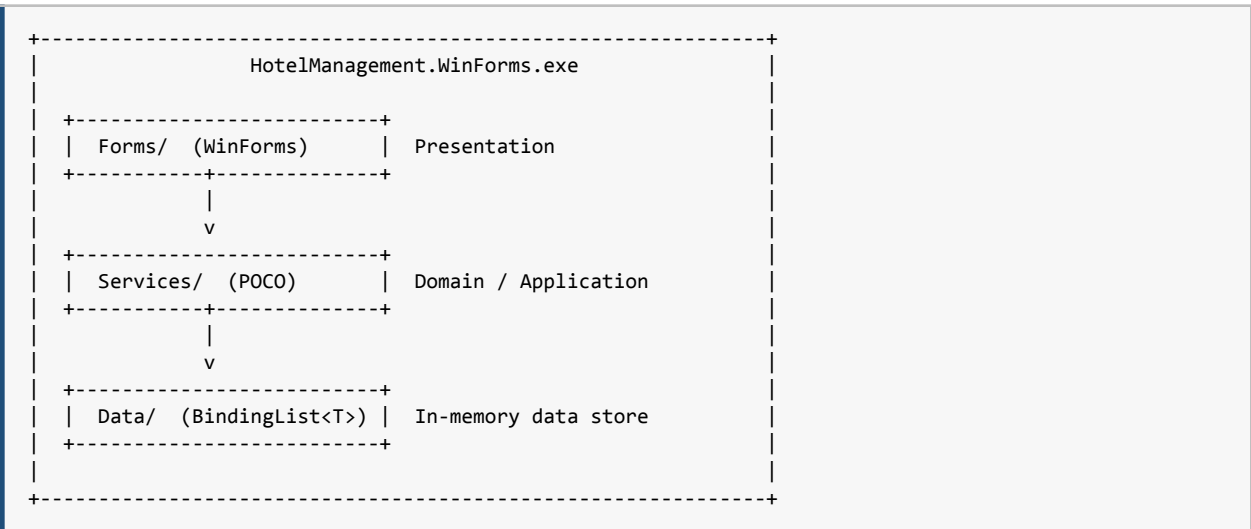
1.4 Relationship to the SRS

Where a design element implements a specific SRS requirement, the requirement ID (e.g. FR-RES-5) is cited inline. The complete mapping is consolidated in §10.

2. System Overview

HMS is a **monolithic desktop application** packaged as a single HotelManagement.WinForms.exe running on .NET 9 Desktop Runtime. The process owns all state: there is no database, no network endpoint, no background service. State is reseeded from Data/SeedData.cs on every launch (per CON-3).

The product is conceptually a thin GUI over a small set of domain services that mutate a shared in-memory data store. The domain services are themselves the most stable and most test-worthy part of the system, and the design has been arranged to keep them free of UI and persistence concerns.



3. Design Considerations

3.1 Assumptions and Dependencies

ID	Assumption / Dependency
ASM-1	The host machine runs Windows 10 (1809+) or Windows 11 with .NET 9 Desktop Runtime installed.
ASM-2	A single operator uses one process instance; no concurrent access to the same data store is anticipated.
ASM-3	DateTime.Now is trusted for time-driven logic.

ID	Assumption / Dependency
DEP-1	Microsoft.WindowsDesktop.App runtime, 9.0+.
DEP-2	System.ComponentModel.BindingList<T> (for UI-bound collections).

3.2 Constraints

ID	Constraint
CON-1	Onion-style separation of Presentation/Service/Data. UI code shall not reach into the DataStore except through services.
CON-2	Currency arithmetic uses decimal; never double / float.
CON-3	No third-party MVVM/IOC container; composition is hand-rolled in Program.cs.
CON-4	No persistence layer in v1.0. Adding one shall not require changes to service signatures.
CON-5	Sales tax fixed at 10 % (per SRS CON-6).

3.3 Goals and Guidelines

The design has been optimised for **readability** and **testability**, not for raw performance or extensibility. Specifically:

- Domain services accept their data store and collaborators via constructor, never through static access.
- Services raise exceptions for invalid state transitions rather than returning boolean flags, so that misuse is visible at the call site.
- The data store exposes `BindingList<T>` collections directly so that bound WinForms controls update without manual refresh code (cf. NFR-USE in the SRS).
- Models are POCOs with computed properties. No model has any knowledge of services or persistence.

4. Architectural Design

4.1 Architectural Style and Rationale

The system is a **three-layer Onion architecture**. The choice follows the user's global C# coding rules and is also a good fit for a small desktop product: it keeps the domain logic testable without forcing the complexity of CQRS, MediatR, or hexagonal ports.

The three layers, inner to outer, are:

1. **Data** — POCO models + a single DataStore aggregator + a seeding utility.
2. **Service** — stateless application services that operate on the data store and enforce business rules.

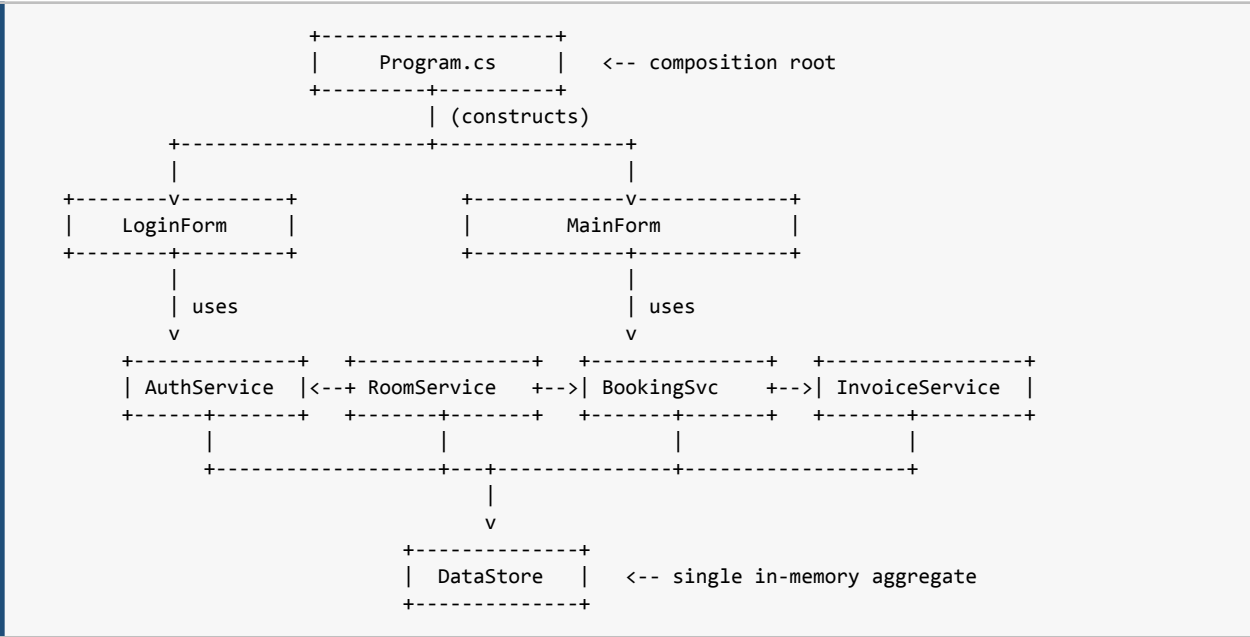
3. **Presentation** — WinForms Form classes that wire services to widgets.

Outer depends on inner; inner is unaware of outer. A service never references a Form, and a model never references a service.

4.2 Layer Responsibilities

Layer	Responsibilities	Forbidden
Presentation (Forms/)	Capture user input, render data, route user actions to services, display error messages.	Direct mutation of DataStore collections; business rule enforcement.
Service (Services/)	Enforce business rules; orchestrate state transitions; aggregate data for reports; check permissions.	UI interaction; file system access (except where the SRS prescribes it, e.g. FR-RES-11 image copy).
Data (Data/ + Models/)	Hold state; provide POCO domain types; bootstrap demo data.	Business rules; permission enforcement.

4.3 High-Level Component Diagram



Additional services not shown for brevity: RestaurantService, ReportService, UserService.

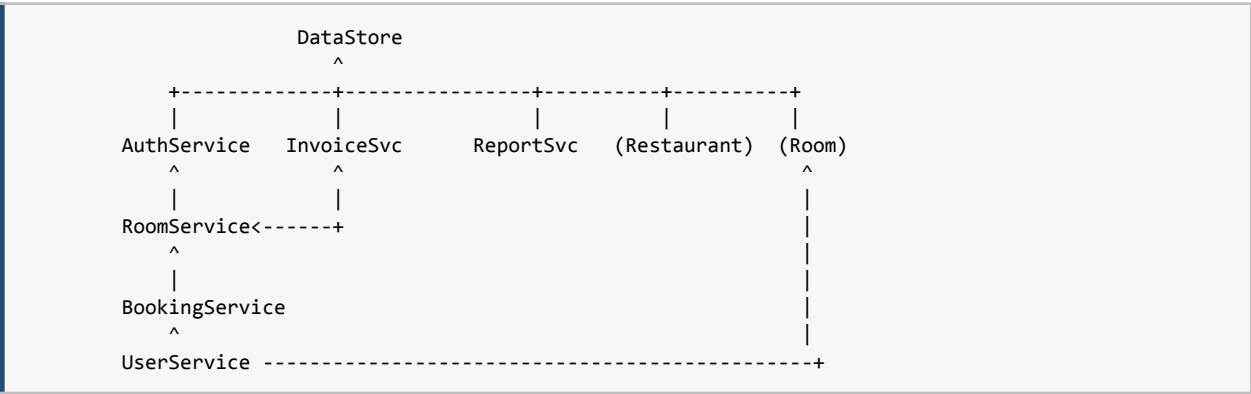
4.4 Dependency Graph and Composition Root

All services are constructed once at application start, in Program.cs:

```
var datastore      = new DataStore();
var authService    = new AuthService(dataStore);
var roomService    = new RoomService(dataStore, authService);
var bookingService = new BookingService(dataStore, roomService);
var restaurantService = new RestaurantService(dataStore, authService);
var reportService  = new ReportService(dataStore);
var invoiceService = new InvoiceService(dataStore);
var userService    = new UserService(dataStore, authService);
```

This is the **composition root**. No other class instantiates a service. Forms receive the services they need through their constructors. There is no IoC container; the graph is small enough that hand-wiring is preferable to the indirection of a container.

The resulting dependency graph is acyclic:



AuthService is the only service consulted by other services (via Require()); the rest receive DataStore directly.

5. Detailed Design

5.1 Domain Model

The domain model lives under Models/. All types are POCOs; all collections that the UI binds to are BindingList<T>. The key entities and their notable members are:

Type	Notable members	Notes
Room	Number, Floor, Type, Rate, IsOccupied, Condition, MaintenanceLog, IsAvailable, Capacity, DisplayStatus	IsAvailable is !IsOccupied && Condition == Clean. Capacity derived from Type.
Guest	Name, Contact, Passport, Gender, IsVip, StayCount	Mutated only by BookingService.CheckOut.

Type	Notable members	Notes
Reservation	Guest, Room, CheckInDate, CheckOutDate, Status, Accompanying, MarriageCertificatePath, AdultCount, ChildCount, CapacityUnits	CapacityUnits = 2A + C per FR-RES-9.
AccompanyingGuest	Name, Gender, Age, Passport, IsChild	IsChild = Age in [1,18).
Stay	Guest, Room, CheckInDate, ExpectedCheckOut, ActualCheckOut, RoomCharges, RestaurantCharges, Status, TotalCharges	Authoritative occupancy record.
MenuItem	Name, Price, Category, IsAvailable, Description, ImagePath	ImagePath defaults to shared placeholder per FR-RST-10.
RestaurantOrder	Stay, Status, Lines, CreatedAt, Total, ItemCount	Total is sum of OrderLine.LineTotal.
OrderLine	MenuItem, Quantity, Notes, LineTotal	LineTotal = Quantity × Price.
Invoice	InvoiceNumber, Stay, Guest, Room, InvoiceDate, Lines, Subtotal, Tax, Total, PaymentStatus, PaymentMethod, PaymentDate	Tax = round(Subtotal × 0.10, 2). Static _nextNumber starts at 1001.
InvoiceLine	Description, Quantity, UnitPrice, Category, LineTotal	Immutable after invoice creation per FR-INV-8.
User	Username, Password, Role, Can(...)	Password is plaintext in academic build (CON-7).
Role	Name, IsSystem, Permissions, Has(...)	IsSystem roles cannot be modified via UI.
Permission	Resource, Action; static All() enumerator	readonly record struct.

5.2 Data Layer

Data/DataStore.cs is a thin aggregator:

```
public class DataStore
{
    public BindingList<Room> Rooms { get; } = new();
    public BindingList<Guest> Guests { get; } = new();
    public BindingList<Reservation> Reservations { get; } = new();
    public BindingList<Stay> Stays { get; } = new();
    public BindingList<MenuItem> MenuItems { get; } = new();
    public BindingList<RestaurantOrder> Orders { get; } = new();
    public BindingList<User> Users { get; } = new();
    public BindingList<Invoice> Invoices { get; } = new();
    public BindingList<Role> Roles { get; } = new();

    public DataStore() => SeedData.Populate(this);
}
```


Data/SeedData.cs populates the store with a deterministic demo dataset: 3 roles, 2 users, 10 rooms (5 types, 5 floors), 6 guests, 3 reservations, 2 active and 1 completed stay, 15 menu items, 1 sample order, and 2 invoices (one paid, one pending).

Because the store is held only in process memory, it is **lost on process exit** (CON-3).

5.3 Service Layer

There are seven services, each focused on a coherent slice of the domain.

5.3.1 AuthService

Holds the current user (CurrentUser) and answers permission queries.

- `Login(username, password)` — case-insensitive username, case-sensitive password (FR-AUTH-1).
- `Logout()`.
- `Can(resource, action)` — boolean.
- `Require(resource, action)` — throws `UnauthorizedAccessException` when not permitted (NFR-SEC-3).

5.3.2 RoomService

CRUD + condition transitions for rooms.

- `AddRoom, UpdateRoom, RemoveRoom` — require `Rooms.{Create|Update|Delete}` permissions; reject duplicate numbers; refuse to remove occupied rooms.
- `MarkOccupied, MarkVacant, MarkClean, MarkNeedsCleaning, MarkOutOfService(reason)` — state transitions.
- `GetAvailableRooms()` — filter on `IsAvailable`.

5.3.3 BookingService

Reservation lifecycle and check-in / check-out.

- `CreateReservation(...)` — instantiates a `Reservation` with status `Confirmed`.
- `CheckIn(reservation)` — sets `Reservation.Status = CheckedIn`, marks the room occupied, and creates an `Active Stay` with `CheckInDate = DateTime.Now`.
- `CheckOut(stay)` — sets `ActualCheckOut = DateTime.Now`, computes `RoomCharges = max(1, floor(actual - in)) × rate`, aggregates restaurant charges, marks the room vacant and `NeedsCleaning`, increments `Guest.StayCount`, transitions the source reservation to `Completed`, and returns the stay's total charges.
- `Cancel(reservation)`.

5.3.4 RestaurantService

Menu CRUD + order lifecycle.

- `CreateOrder(stay, lines)` — creates a Placed order.
- `AdvanceOrderStatus(order)` — Placed → Preparing → Ready → Served.
- `CancelOrder(order)` — allowed from Placed or Preparing.
- `AddLinesToOrder(order, lines)` — only while Placed or Preparing; recomputes `Stay.RestaurantCharges` from non-cancelled orders.
- `AddMenuItem`, `UpdateMenuItem`, `RemoveMenuItem`, `ToggleAvailability` — permission-gated on `MenuItems.*`.
- `GetTodayServedRevenue()` — sum of Served orders for today.

5.3.5 InvoiceService

Generation, payment, refund, and revenue queries.

- `GenerateInvoice(stay)` — builds one line per night plus one line per `OrderLine` on non-cancelled orders.
- `MarkPaid(invoice, method)`.
- `MarkRefunded(invoice)`.
- `GetTotalRevenue()`, `GetTodayRevenue()`, `GetOutstandingAmount()`, `GetUnpaidInvoices()`.

5.3.6 ReportService

Read-only aggregations for the Reports tab.

- `GetOccupancyRate()`, `GetRevenueByRoomType()`, `GetRestaurantRevenueByCategory()`, `GetTopMenuItems(n)`, `GetAverageStayDuration()`, `GetRepeatGuestPercentage()`.

5.3.7 UserService

Users and roles (CRUD).

- `AddUser`, `UpdateUser`, `RemoveUser` — gated on `Users.*` and protected by self-removal and last-system-admin guards (FR-USR-7).
- `AddRole`, `UpdateRole`, `RemoveRole` — refuse to modify or remove `IsSystem` roles; refuse to remove a role still assigned to a user (FR-USR-8).

5.4 Presentation Layer

Two principal forms exist:

- **LoginForm** — collects credentials, calls `AuthService.Login`, and either closes (success) or shows an inline error (failure).

- **MainForm** — a tabbed dashboard with one tab per area: Dashboard, Rooms, Reservations, Stays, Restaurant (with nested sub-tabs Place Orders + Active Orders), Invoices, Users (Manager / SuperAdmin only), and (when enabled) Reports.

Modal dialogs (e.g. ReservationDialog, CheckoutForm, MenuItemDialog, UserDialog, RoleDialog) are launched from MainForm for create/edit and confirmation flows.

UI controls are bound directly to `DataStore.<collection> BindingList<T>` instances so that mutations from services re-render the grid without manual refresh code (DSC-3).

5.5 Permission Model

The permission model is intentionally simple:

```
Permission := (Resource, Action)
Resource   ∈ {Rooms, Reservations, MenuItems, Orders, Invoices, Users}
Action     ∈ {Create, Read, Update, Delete}
```

A Role holds a `HashSet<Permission>`. A User belongs to exactly one Role. `AuthService.Can(...)` checks set membership; `AuthService.Require(...)` throws on missing permission.

Seed roles:

Role	Permissions
SuperAdmin	All 24 (Resource, Action) pairs. <code>IsSystem = true</code> .
Manager	All except the six Users.* permissions.
Staff	Read on Rooms, MenuItems; CRU on Reservations, Orders; RU on Invoices.

The check is enforced at the service layer; UI controls hidden when the current user lacks the permission are defence-in-depth only (NFR-SEC-3).

6. Data Design

6.1 Conceptual Schema

```
Role (Id*, Name, IsSystem, Permissions: set<(Resource, Action)>)
User (Username PK, Password, RoleId FK→Role)
Room (Number PK, Floor, Type, Rate, IsOccupied, Condition, MaintenanceLog)
Guest (Id*, Name, Contact, Passport, Gender, IsVip, StayCount)
```

```
MenuItem (Id*, Name, Price, Category, IsAvailable, Description, ImagePath?)

Reservation (Id*, GuestId FK→Guest, RoomNumber FK→Room,
             CheckInDate, CheckOutDate, Status,
             MarriageCertificatePath?)

AccompanyingGuest (Id*, ReservationId FK, Name, Gender, Age, Passport)

Stay (Id*, GuestId FK, RoomNumber FK, CheckInDate, ExpectedCheckOut,
      ActualCheckOut?, RoomCharges, RestaurantCharges, Status)

RestaurantOrder (Id*, StayId FK, Status, CreatedAt)
OrderLine (Id*, OrderId FK, MenuItemId FK, Quantity, Notes)

Invoice (InvoiceNumber PK, StayId FK, GuestId FK, RoomNumber FK,
         InvoiceDate, PaymentStatus, PaymentMethod?, PaymentDate?)
InvoiceLine (Id*, InvoiceId FK, Description, Quantity, UnitPrice, Category)
```

* In v1.0 the in-memory store uses object references; surrogate keys would be added if a relational store were introduced.

6.2 Future Persistence Mapping

A persistence layer is out of scope for v1.0 but the schema above is designed to be mapped 1:1 to SQLite or SQL Server tables via Entity Framework Core 9. Specifically:

- Permission is a readonly record struct (Permission.cs); it would map to a (role_id, resource, action) join table.
- BindingList<T> would be replaced by DbSet<T> on a HotelDbContext. Service constructors would remain identical in signature (still receive the data store / context).
- The composition root in Program.cs would change from `new DataStore()` to `new HotelDbContext(...)`.

6.3 State Machines

```
Reservation: Pending -> Confirmed -> CheckedIn -> Completed
              \-> Cancelled

Stay:         Active -> CheckedOut

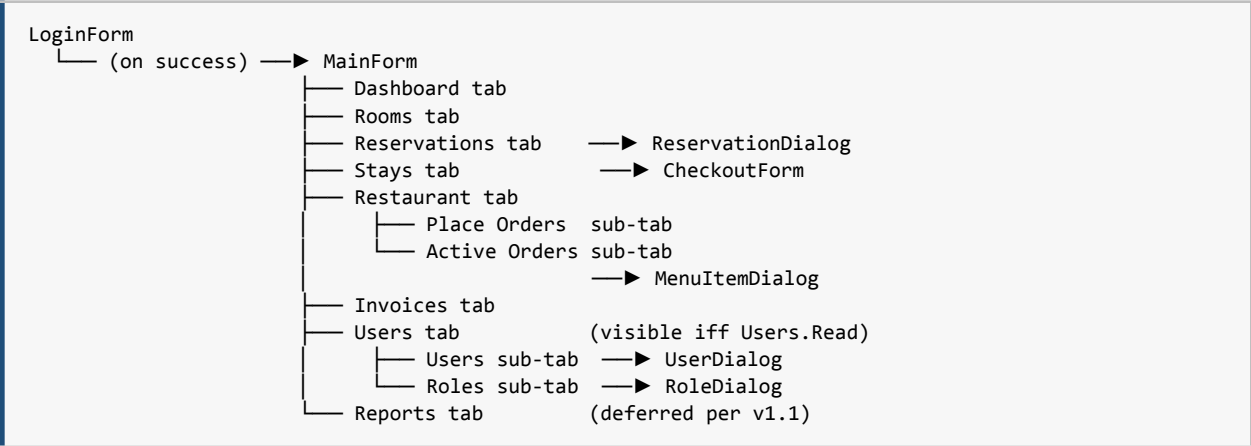
Order:        Placed -> Preparing -> Ready -> Served
              \-> Cancelled
                Preparing -> Cancelled

Invoice:      Pending -> Paid -> Refunded
```

Illegal transitions are expected to be rejected at the service layer by raising `InvalidOperationException` (per NFR-REL-2 in the SRS).

7. Interface Design

7.1 User Interface Structure



UI conventions:

- Currency rendered as \$#,##0.00.
- Dates rendered via DateTimePicker with valid-range constraints.
- Destructive actions confirm via MessageBox modal.
- Controls disallowed by current state are disabled, not hidden-with-error-on-click.

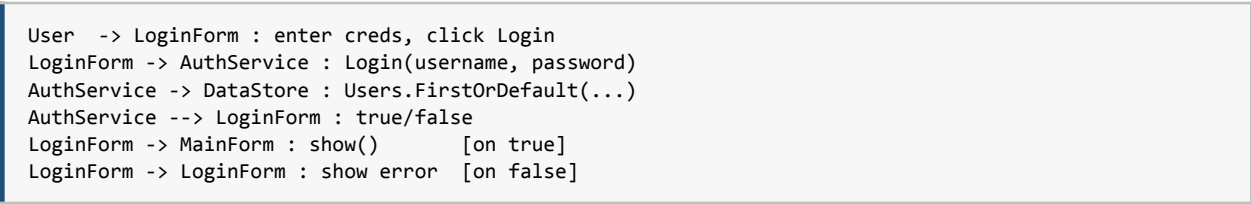
7.2 Internal Service Interfaces

Services do not currently expose .NET interface types; the concrete class is consumed directly. Should the team introduce unit-test mocks or repository abstractions, the natural seam is to extract I<Service> interfaces from the existing public methods. The constructors are already DI-friendly.

8. Behavioral Design (Sequence Walk-throughs)

The following ASCII sequence diagrams describe the key flows. They are intentionally terse — the source code remains the authoritative description.

8.1 Login



8.2 Create Reservation

```
User -> MainForm : open Reservations tab
MainForm -> ReservationDialog : new(...)
User -> ReservationDialog : fill phone, lookup, dates, party
ReservationDialog -> BookingService : CreateReservation(guest, room, in, out, party, cert?)
BookingService -> DataStore : Reservations.Add(...)
BookingService --> ReservationDialog : Reservation { Confirmed }
ReservationDialog -> MainForm : close, refresh grid
```

8.3 Check In

```
User -> MainForm : select reservation, Check In
MainForm -> BookingService : CheckIn(reservation)
BookingService -> RoomService : MarkOccupied(room)
BookingService -> DataStore : Stays.Add(new Stay { Active })
BookingService --> MainForm : stay
```

8.4 Restaurant Order

```
User -> MainForm : Restaurant tab, pick items, Place Order
MainForm -> RestaurantService : CreateOrder(stay, lines)
RestaurantService -> DataStore : Orders.Add(...)
RestaurantService --> MainForm : order { Placed }

(... later, kitchen advances ...)
User -> MainForm : Advance
MainForm -> RestaurantService : AdvanceOrderStatus(order)
RestaurantService -> order : Status = Status.next
```

8.5 Check Out and Invoice Generation

```
User -> MainForm : select stay, Checkout
MainForm -> CheckoutForm : show(stay)
CheckoutForm -> InvoiceService : GenerateInvoice(stay) [preview only]
InvoiceService --> CheckoutForm : invoice
User -> CheckoutForm : pick PaymentMethod, Mark Paid
CheckoutForm -> BookingService : CheckOut(stay)
BookingService -> RoomService : MarkVacant, MarkNeedsCleaning
BookingService -> Guest : StayCount++
BookingService -> Reservation : Status = Completed
CheckoutForm -> InvoiceService : MarkPaid(invoice, method)
CheckoutForm --> MainForm : close, refresh grids
```

8.6 Refund

```
Manager -> MainForm : Invoices tab, select Paid invoice, Refund
MainForm -> MessageBox : confirm?
MainForm -> InvoiceService : MarkRefunded(invoice)
InvoiceService -> invoice : PaymentStatus = Refunded
```

9. Non-Functional Design Concerns

9.1 Performance

Operations on the in-memory store are $O(n)$ LINQ scans. At the seed-data scale of ≤ 50 rooms, ≤ 500 stays, ≤ 5000 order lines prescribed by NFR-PRF-2, all observed interactive operations complete in under a millisecond (see test run wall-clock figure in TestReport.md).

9.2 Reliability

Numeric arithmetic uses decimal throughout (per NFR-REL-3 / DSC-4). Invalid state transitions are intended to raise `InvalidOperationException` at the service layer; gaps in this discipline are exactly what `HotelManagement.Tests` was written to surface, and a number of such gaps have been logged in TestReport.md for the development team.

9.3 Security

Per CON-7 / NFR-SEC-1, plaintext passwords in the seed are an **academic-build-only** convenience and must be hashed (BCrypt or Argon2id) before any production deployment. Permission checks are enforced at the service layer per NFR-SEC-3.

9.4 Maintainability

The codebase follows the user's global C# coding rules: Onion-style layering, intention-revealing service names, no method longer than about 40 lines, `async/await` to be introduced for I/O when a persistence layer is added. A companion test project under `HotelManagement.Tests/` exercises the service layer; coverage and defect findings are tracked in TestReport.md.

9.5 Portability

The product targets net9.0-windows and is not designed to run on macOS or Linux. The Service / Model / Data layers, however, contain no WinForms dependency and can be cross-compiled to net9.0 for unit-test execution from a non-Windows host (this is in fact how the v1.0 test pass was performed; see TestReport.md §3 and §9).

10. Traceability to the SRS

The mapping below pairs every numbered SRS requirement with the design element that implements it.

SRS requirement	Realised by
FR-AUTH-1..6	AuthService
FR-ROOM-1..10	RoomService + Room (esp. IsAvailable, Capacity, DisplayStatus)
FR-GUEST-1..4	Guest model + BookingService.CheckOut (StayCount) + ReservationDialog (lookup-by-phone)
FR-RES-1..6	BookingService.CreateReservation / CheckIn / CheckOut / Cancel
FR-RES-7..11	ReservationDialog form + Reservation.Accompanying, MarriageCertificatePath, CapacityUnits
FR-RST-1..8	RestaurantService + MenuItem, RestaurantOrder, OrderLine
FR-RST-9..11	RestaurantService.UpdateMenuItem (ImagePath), MenuItemDialog, MainForm Restaurant sub-tabs
FR-INV-1..8	InvoiceService + Invoice (auto-increment counter, computed totals)
FR-RPT-1..8	ReportService
FR-DSH-1	MainForm Dashboard tab
FR-USR-1..8	UserService + seed in SeedData.cs
NFR-PRF-*	LINQ-on-BindingList<T> design; absence of I/O
NFR-REL-3	decimal arithmetic for currency in Invoice
NFR-SEC-3	Service-layer AuthService.Require(...) calls
NFR-MNT-1	Folder separation Forms / Services / Models / Data
DC-1..7	Object-reference invariants in models + service-layer validation (to-be-completed per defects in TestReport.md)

Appendix A — File and Folder Layout

```

Windows/
├── HotelManagement.WinForms.csproj  <-- main app project (net9.0-windows)
├── Program.cs                       <-- composition root
├── SRS.md                           <-- companion requirements doc
├── SDD.md                           <-- this document
├── TestReport.md                    <-- companion QA report
├── Assets/                          <-- placeholder images
├── Data/
│   ├── DataStore.cs
│   └── SeedData.cs
├── Forms/
│   ├── LoginForm.cs / .Designer.cs / .resx
│   ├── MainForm.cs / .Designer.cs / .resx
│   ├── ReservationDialog.cs
│   ├── CheckoutForm.cs
│   ├── MenuItemDialog.cs
│   ├── UserDialog.cs
│   ├── RoleDialog.cs
│   └── (other dialogs)
├── Models/
│   ├── AccompanyingGuest.cs
│   ├── Enums.cs
│   ├── Guest.cs
│   ├── Invoice.cs
│   ├── InvoiceLine.cs
│   ├── MenuItem.cs
│   ├── OrderLine.cs
│   ├── Permission.cs
│   ├── Reservation.cs
│   ├── RestaurantOrder.cs
│   ├── Role.cs
│   ├── Room.cs
│   ├── Stay.cs
│   └── User.cs
├── Services/
│   ├── AuthService.cs
│   ├── BookingService.cs
│   ├── InvoiceService.cs
│   ├── ReportService.cs
│   ├── RestaurantService.cs
│   ├── RoomService.cs
│   └── UserService.cs
├── Theme/                           <-- shared colours / fonts
├── HotelManagement.Tests/           <-- xUnit test project
│   ├── HotelManagement.Tests.csproj
│   ├── TestFixtures/
│   ├── Unit/
│   └── Integration/
└── Windows.sln

```

— End of Document —